

Java coding programs

Array

```
package Array;

public class Array {

    public static void main(String[] args) {

        int arr[] = {10, 20, 30, 40, 50};

        for(int i = 0; i < arr.length; i++){
            System.out.println(arr[i]);
        }
    }
}
```

Arraytypes

```
package Array;

import java.util.Scanner;
public class Arraytypes {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        // create an array
        char data[] = new char[6];

        data[0] = 'j';
        data[1] = 'i';
        data[2] = 's';
        data[3] = 'h';
        data[4] = 'a';
        data[5] = 'n';

        for(int i = 0; i < data.length; i++){
            System.out.println(data[i]);
        }

        input.close();
    }
}
```

AverageofArray

```
package Array;
import java.util.Scanner;

public class AverageOfArray {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        System.out.println("Enter the Size of the array : ");
        int size = input.nextInt();

        int arr[] = new int[size];

        for(int i = 0; i < arr.length; i++){
            System.out.println("Enter the num " + (i+1));
            arr[i] = input.nextInt();
        }

        int sum = 0;
        for(int i = 0; i < arr.length; i++){
            sum = sum + arr[i];
        }
        System.out.println("The Sum of the array is : " + sum);

        double avg = (sum / size);
        System.out.println("The Average of the Array is : " + avg);

        // for(int i = 0; i<= arr.length; i++){
        //     System.out.println(arr[i]);
        // }
        input.close();
    }
}
```

CountFrequencyOfElement

```
package Array;
import java.util.HashMap;
import java.util.Map;

public class CountFrequencyOfElement {
```

```
public static void main(String[] args) {

    int arr[] = {10, 20, 10, 40, 20, 30, 50};

    Map<Integer, Integer> hm = new HashMap<Integer, Integer>();

    for(int i = 0; i<arr.length; i++){

        if(hm.containsKey(arr[i])){
            hm.put(arr[i], hm.get(arr[i]) + 1);
            //(key, value + 1)
        }
        else{
            hm.put(arr[i], 1);
        }

        // Another simple way for without If block
        // hm.put(arr[i], hm.getOrDefault(arr[i], 0) + 1);
    }

    for(Map.Entry<Integer, Integer> ele : hm.entrySet()){
        System.out.println("Element = " + ele.getKey() + " Frequency = " +
        ele.getValue());
    }
}
```

FindDublicates

```
package Array;

public class FindDublicates {

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 2, 5, 6, 3, 7};

        System.out.println("Duplicate elements:");

        // Outer loop: Traverse each element in the array
        for (int i = 0; i < arr.length; i++) {
            boolean isDuplicate = false; // Assume the current element is not a
duplicate

            // Inner loop: Check if the current element (arr[i]) is found later in
the array
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[i] == arr[j]) { // If a duplicate is found
                    isDuplicate = true; // Mark as duplicate
                    break; // No need to check further; exit the inner loop
                }
            }
        }
    }
}
```

```

    }

    // If a duplicate was found, print the element
    if (isDuplicate) {
        System.out.println(arr[i]);
    }
}
}

```

FindkthLargestEle

```

package Array;

public class FindkthLargestEle{

    public static int findkthLargest(int[] arr, int left, int right, int k) {
        if (left == right) {
            return arr[left]; // Only one element left
        }

        // Partition the array and get the pivot index
        int pivotIndex = Partition(arr, left, right);

        // If pivot is the kth smallest element, return it
        if (pivotIndex == k - 1) {
            return arr[pivotIndex];
        }
        // If k is smaller, search in left half
        else if (pivotIndex > k - 1) {
            return findkthLargest(arr, left, pivotIndex - 1, k);
        }
        // If k is larger, search in right half
        else {
            return findkthLargest(arr, pivotIndex + 1, right, k);
        }
    }

    public static int Partition(int arr[], int left, int right) {
        int pivot = arr[right]; // Choose the last element as pivot
        int i = left; // Initialize index for smaller elements

        for (int j = left; j < right; j++) {
            if (arr[j] >= pivot) {
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
                i++;
            }
        }
    }
}

```

```
// Place pivot in its correct position
int temp = arr[i];
arr[i] = arr[right];
arr[right] = temp;

return i; // Return pivot index
}

public static void main(String[] args) {

    int[] arr = {10, 80, 30, 90, 40, 50, 70};
    int k = 3;

    int kthLargest = findkthLargest(arr, 0, arr.length-1, k);
    System.out.println(k + "th kthLargest element is: " + kthLargest);
}
}
```

FindkthSmallestEle

```
package Array;

public class FindkthSmallestEle{

    public static int findKthSmallest(int[] arr, int left, int right, int k) {
        if (left == right) {
            return arr[left]; // Only one element left
        }

        // Partition the array and get the pivot index
        int pivotIndex = Partition(arr, left, right);

        // If pivot is the kth smallest element, return it
        if (pivotIndex == k - 1) {
            return arr[pivotIndex];
        }
        // If k is smaller, search in left half
        else if (pivotIndex > k - 1) {
            return findKthSmallest(arr, left, pivotIndex - 1, k);
        }
        // If k is larger, search in right half
        else {
            return findKthSmallest(arr, pivotIndex + 1, right, k);
        }
    }

    public static int Partition(int arr[], int left, int right) {
        int pivot = arr[right]; // Choose the last element as pivot
        int i = left; // Initialize index for smaller elements
    }
}
```

```
        for (int j = left; j < right; j++) {
            if (arr[j] <= pivot) {
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
                i++;
            }
        }

        // Place pivot in its correct position
        int temp = arr[i];
        arr[i] = arr[right];
        arr[right] = temp;

        return i; // Return pivot index
    }

    public static void main(String[] args) {

        int[] arr = {10, 80, 30, 90, 40, 50, 70};
        int k = 3;

        int kthSmallest = findKthSmallest(arr, 0, arr.length-1, k);
        System.out.println(k + "th smallest element is: " + kthSmallest);
    }
}
```

FindMaxAndMin

```
package Array;

public class FindMaxAndMin {

    public static void main(String[] args) {

        int arr[] = {1, 2, 3, -1, 5, 6, -2, 7};

        int max= arr[0];
        int min = arr[0];

        for(int i = 1; i < arr.length; i++){
            if(arr[i] > max){
                max = arr[i];
            }

            if(arr[i] < min){
                min = arr[i];
            }
        }
    }
}
```

```
        System.out.println(max);
        System.out.println(min);
    }
}
```

JaggedArray

```
package Array;

public class JaggedArray {

    public static void main(String[] args) {

        // First Way to create a jagged Array
        // Here row Size is known but column Size is unknown
        // int arr[][] = {
        //     {1, 2, 3},
        //     {4,5},
        //     {6}
        // };

        // System.out.println(arr[2][0]);

        // Second Way
        char arr1[][] = new char[4][] ;

        arr1[0] = new char[1];    // 1 is column size
        arr1[1] = new char[2];    // 2 is column size
        arr1[2] = new char[3];
        arr1[3] = new char[4];

        arr1[0][0] = 'A';

        arr1[1][0] = 'B';
        arr1[1][1] = 'C';

        arr1[2][0] = 'D';
        arr1[2][1] = 'E';
        arr1[2][2] = 'F';

        arr1[3][0] = 'G';
        arr1[3][1] = 'H';
        arr1[3][2] = 'I';
        arr1[3][3] = 'J';

        // System.out.println(arr1[3][1]);

        for(int i = 0; i < arr1.length; i++){
            for(int j= 0; j < arr1[i].length; j++){
```

```
        System.out.print(arr1[i][j] + " ");
    }
    System.out.println();
}

// Using forEach loop
// for(char[] ans : arr1){
//     System.out.println(ans);
// }
}
```

MergeArray

```
package Array;

import java.util.Arrays;
import java.util.Collections;

public class MergeArray {

    public static Integer[] mergeArray(int arrA[], int arrB[]){

        Integer finalmergearray[] = new Integer[arrA.length + arrB.length];
        int k = 0;

        // for traversing arrA element
        for(int i = 0; i < arrA.length; i++){
            finalmergearray[k] = arrA[i];
            k++;
        }

        // for traversing arrB element
        for(int j = 0; j < arrA.length; j++){
            finalmergearray[k] = arrB[j];
            k++;
        }

        Arrays.sort(finalmergearray, Collections.reverseOrder());
        return finalmergearray;
    };

    public static void main(String[] args) {

        int arrA[] = {5, 20, 89, 1, -7};
        int arrB[] = {32, 58, 2, 50, 18};

        System.out.println(Arrays.toString(arrA));
        System.out.println(Arrays.toString(arrB));
    }
}
```



```
        Integer mergedArray[] = mergeArray(arrA, arrB);

        System.out.println(Arrays.toString(mergedArray));
    }
}
```

MergeTwoSortedArray

```
package Array;

public class MergeTwoSortedArray {

    public static int[] MergeTwoSorted(int arr1[], int arr2[]){

        int mergearray[] = new int[arr1.length + arr2.length];

        int i = 0; int j = 0; int k = 0;

        // Merge elements in sorted order
        while(i < arr1.length && j < arr2.length){
            if(arr1[i] <= arr2[j]){
                mergearray[k] = arr1[i];
                k++; i++;
            }
            else{ // if arr1[i] >= arr2[j]
                mergearray[k] = arr2[j];
                k++; j++;
            }
        }

        // Copy remaining elements of arr1 (if any)
        while(i < arr1.length){
            mergearray[k] = arr1[i];
            k++; i++;
        }

        // Copy remaining elements of arr2 (if any)
        while(j < arr2.length){
            mergearray[k] = arr2[j];
            k++; j++;
        }

        return mergearray;
    }

    public static void main(String[] args) {

        int arr1[] = {1, 3, 5, 7};
        int arr2[] = {2, 4, 6, 8, 9};

        int merge[] = MergeTwoSorted(arr1, arr2);
    }
}
```

```
        for(int num : merge){
            System.out.print(num + " ");
        }
    }
}
```

MergeTwoUnsortedArray

```
package Array;

import java.util.Arrays;

public class MergeTwoUnsortedArray {

    public static int[] mergeArray(int arr1[], int arr2[]){

        int finalmergearray[] = new int[arr1.length + arr2.length];
        int k = 0;

        // for travsersing arrA element
        for(int i = 0; i < arr1.length; i++){
            finalmergearray[k] = arr1[i];
            k++;
        }

        // for travsersing arrB element
        for(int j = 0; j < arr2.length; j++){
            finalmergearray[k] = arr2[j];
            k++;
        }

        Arrays.sort(finalmergearray);
        return finalmergearray;
    };

    public static void main(String[] args) {

        int arr1[] = {1, 3, 5, 7};
        int arr2[] = {2, 4, 6, 8, 9};

        int merge[] = mergeArray(arr1, arr2);

        for(int num : merge){
            System.out.print(num + " ");
        }
    }
}
```

MergeTwoUnsortedArray2

```
package Array;

public class MergeTwoUnsortedArray2 {

    public static int[] mergeArray2(int arr1[], int arr2[]){

        int merge[] = new int[arr1.length + arr2.length];
        int k = 0;

        // for traversing arrA element
        for(int num : arr1){
            merge[k++] = num;
        }

        // for traversing arrB element
        for(int num : arr2){
            merge[k++] = num;
        }

        // simple selection sort we can also used Bubble sort
        for(int i = 0; i < merge.length; i++){
            for(int j= i+1; j < merge.length; j++){
                if(merge[j] < merge[i]){
                    int temp = merge[i];
                    merge[i] = merge[j];
                    merge[j] = temp;
                }
            }
        }

        return merge;
    };

    public static void main(String[] args) {

        int arr1[] = {1, 3, 5, 7};
        int arr2[] = {2, 4, 6, 8, 9};

        int mergearray[] = mergeArray2(arr1, arr2);

        for(int num : mergearray){
            System.out.print(num + " ");
        }
    }
}
```

RemoveDuplicatesSorted

```
package Array;

public class RemoveDuplicatesSorted {

    public static int removeDuplicates(int[] arr) {

        int i = 0; // Slow pointer (tracks unique elements)

        for (int j = 1; j < arr.length; j++) {
            if (arr[j] != arr[i]) { // Found a new unique element
                i++;
                arr[i] = arr[j]; // Place it in the correct position
            }
        }

        return i + 1; // New length of array with unique elements
    }

    public static void main(String[] args) {

        int[] arr = {1, 1, 2, 2, 3, 4, 4, 5};

        int newLength = removeDuplicates(arr);

        // Print the unique elements
        System.out.print("Array after removing duplicates: ");
        for (int i = 0; i < newLength; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

RemoveDuplicatesUnSorted

```
package Array;

import java.util.HashSet;

public class RemoveDuplicatesUnSorted {

    public static int removeDuplicates(int[] arr) {

        // To track seen elements
        HashSet<Integer> s = new HashSet<>();

        // To maintain the new size of the array
        int idx = 0;

        for (int i = 0; i < arr.length; i++) {
```

```
        if (!s.contains(arr[i])) {
            s.add(arr[i]);
            arr[idx++] = arr[i];
        }
    }

    // Return the size of the unique elements array
    return idx;
}

public static void main(String[] args) {

    int[] arr = {1, 2, 2, 3, 4, 4, 4, 5, 5};
    int newSize = removeDuplicates(arr);

    for (int i = 0; i < newSize; i++) {
        System.out.print(arr[i] + " ");
    }

}
}
```

ReverseAnArray

```
package Array;

public class ReverseAnArray {

    public static void reverseAnArray(int arr[]){

        int s = 0;
        int e = arr.length - 1;

        while(s < e){

            int temp = arr[s];
            arr[s] = arr[e];
            arr[e] = temp;

            s++;
            e--;
        }

    }

    public static void printarray(int arr[]){

        for(int i= 0; i<arr.length; i++){
            System.out.println(arr[i]);
        }

    }

}
```

```
    }  
    public static void main(String[] args) {  
  
        int arr[] = {10, 20, 30, 40, 50, 60};  
        reverseAnArray(arr);  
        printarray(arr);  
    }  
}
```

SecondSmallestElement

```
package Array;  
  
public class SecondSmallestElement{  
  
    public static int findSecondLargest(int[] arr) {  
  
        int largest = Integer.MIN_VALUE;  
        int secondLargest = Integer.MIN_VALUE;  
  
        for (int num : arr) {  
            if (num > largest) {          // finding largest number  
                secondLargest = largest;  
                largest = num;  
            } else if (num > secondLargest && num != largest) { // finding  
secondlargest number  
                // first condition for num is greater  
than secondLargest and second to handle duplicate largest numbers)  
                secondLargest = num;  
            }  
        }  
  
        return secondLargest;  
    }  
  
    public static void main(String[] args) {  
        int[] arr = {10, 20, 4, 45, 99, 99, 6};  
        System.out.println("Second largest element: " + findSecondLargest(arr));  
    }  
}
```

SecondSmallestElementMethod2

```
package Array;
```

```
public class SecondSmallestElementMethod2 {

    public static int SecondSmall(int arr[]){

        // In Ascending Order
        for(int i= 0; i<arr.length; i++){
            for(int j=i+1; j<arr.length; j++){
                if(arr[j] < arr[i]){
                    int temp = arr[i];
                    arr[i] = arr[j];
                    arr[j] = temp;
                }
            }
        }
        return arr[arr.length - 2];
    }
    public static void main(String[] args) {

        int arr[] = {20, 10, 40, 50, 5};

        int ele = SecondSmall(arr);
        System.out.println("SecondSmallestElement : " + ele);

    }
}
```

SortAnArray

```
package Array;

public class SortAnArray {

    public static void main(String[] args) {

        int arr[] = {20, 10, 40, 50, 5};

        // In Ascending Order
        for(int i= 0; i<arr.length; i++){
            for(int j=i+1; j<arr.length; j++){
                if(arr[j] < arr[i]){
                    int temp = arr[i];
                    arr[i] = arr[j];
                    arr[j] = temp;
                }
            }
        }

        // In Decinding Order
        // for(int i= 0; i<arr.length; i++){
        //     for(int j=i+1; j<arr.length; j++){
```

```
        //          if(arr[j] > arr[i]){
        //              int temp = arr[i];
        //              arr[i] = arr[j];
        //              arr[j] = temp;
        //          }
        //      }
        //  }

        for(int ele : arr){
            System.out.println(ele);
        }
    }
}
```

SumOfArray

```
package Array;

import java.util.Scanner;
public class SumOfArray {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        System.out.println("Enter the Size of the array : ");
        int size = input.nextInt();

        int arr[] = new int[size];

        // taking input from the user
        for(int i = 0; i<arr.length; i++){
            System.out.println("Enter the Num :" + (i+1));
            arr[i] = input.nextInt();
        }

        int sum = 0;
        for(int i = 0; i < arr.length; i++){
            sum = sum + arr[i];
        }

        System.out.println("Sum = " + sum);

        input.close();
    }
}
```


TwoDArrayWithUserInput

```
package Array;

import java.util.Scanner;

public class TwoDArrayWithUserInput {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        System.out.println("Enter the size of the row : ");
        int rowsize = input.nextInt();

        System.out.println("Enter the size of the col : ");
        int colsize = input.nextInt();

        int arr[][] = new int[rowsize][colsize];

        // UserInput
        for (int i = 0; i < rowsize; i++) {
            for (int j = 0; j < colsize; j++) {
                System.out.print("Enter the elements " + i + j + "\n");
                arr[i][j] = input.nextInt();
            }
            System.out.println();
        }

        for (int i = 0; i < rowsize; i++) {
            for (int j = 0; j < colsize; j++) {

                System.out.print(arr[i][j] + " ");
            }
            System.out.println();
        }

        input.close();
    }
}
```

TwoDimentionalArray

```
package Array;

public class TwoDimentionalArray {
```

```
public static void main(String[] args) {  
  
    String[][] arr = new String[3][3];  
  
    arr[0][0] = "Ram";  
    arr[0][1] = "Sham";  
    arr[0][2] = "Raju";  
  
    arr[1][0] = "Baburao";  
    arr[1][1] = "Snehal";  
    arr[1][2] = "Samarth";  
  
    arr[2][0] = "Neha";  
    arr[2][1] = "Manjiri";  
    arr[2][2] = "Aditya";  
  
    for(int i= 0; i < arr.length; i++){  
        for(int j= 0; j < arr.length; j++){  
  
            System.out.print(arr[i][j] + " ");  
  
        }  
        System.out.println();  
    }  
}
```